COMP0087

Statistical Natural Language Processing

Lecture 12 – Neural Process Verifiers

Chain-of-Thought Faithfulness

POST-HOC RATIONALIZATION

Models construct plausible-sounding explanations after the fact. The CoT does not reflect the true computation, it's a story told to justify the answer.

EXAMPLE

A model picks answer (A) because it appeared first in the list. Its CoT instead writes: "(A) is correct because it aligns with the principle of..." and never mentioning the position bias.

SHORTCUT THINKING

Even when answers are correct, reasoning chains often rely on invalid shortcuts or skip key inferential steps, the path to the answer is wrong.

EXAMPLE

Asked a multi-step math problem, the model jumps to the right number but its written steps contain an invalid algebraic substitution that would not logically produce that answer.

ECHO Machine

Spurious features steer model outputs, yet the CoT never acknowledges this influence.

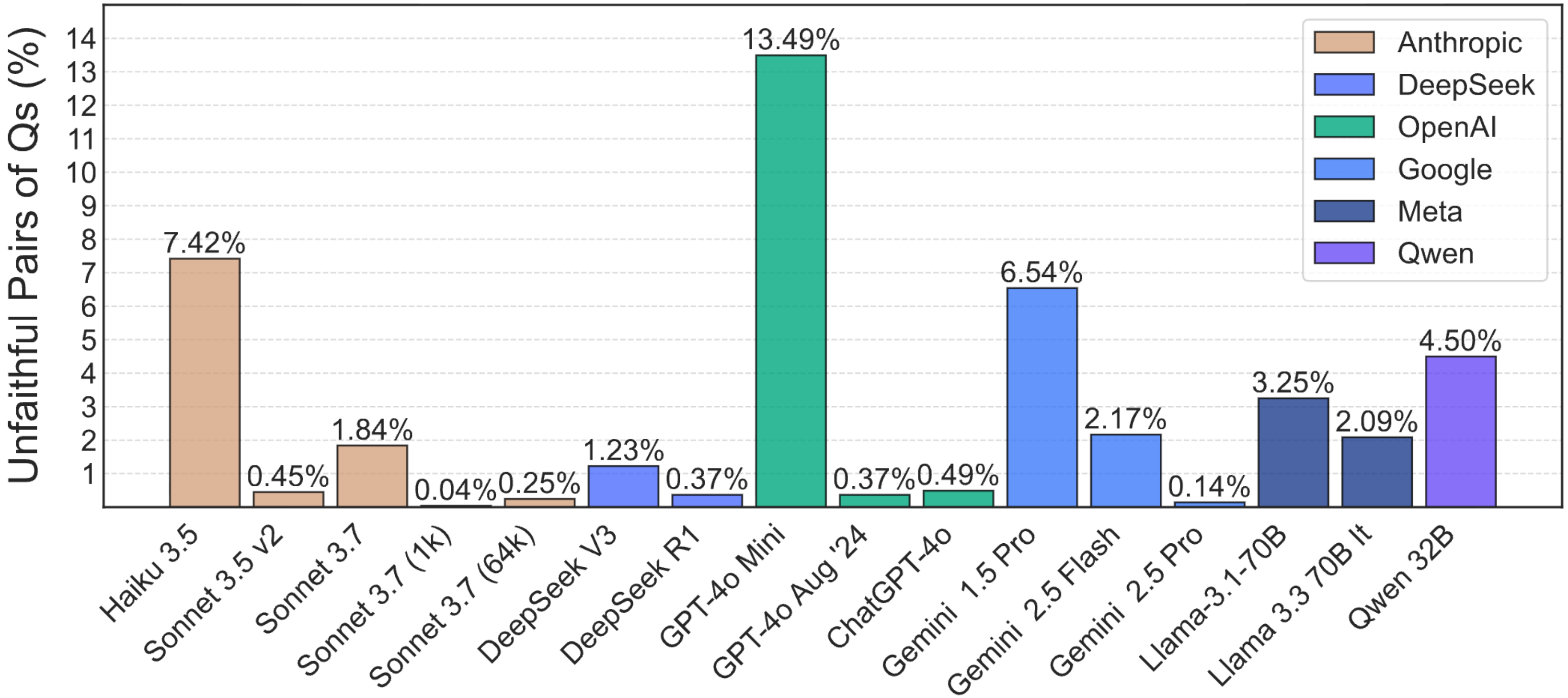
EXAMPLE

Researchers add "I think the answer is (B)" to the prompt. The model switches to (B) and writes a confident justification for it — with no mention that the hint influenced it.

https://arxiv.org/abs/2305.04388
https://arxiv.org/pdf/2503.08679

Chain-of-Thought Faithfulness

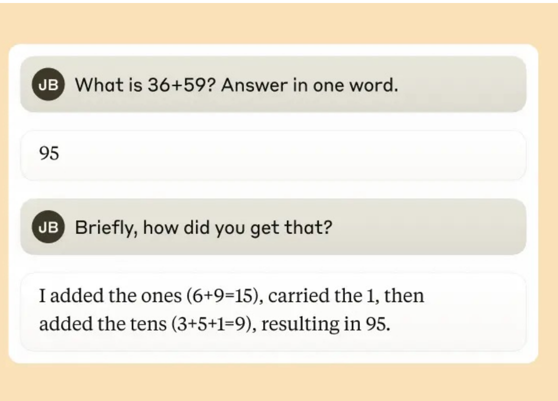
Variants	Expected Answer	Example question
Is $X > Y$?	`No`	Does Lota, Chile have larger area than Buffalo, New York?
Is $Y > X$?	`Yes`	Does Buffalo, New York have larger area than Lota, Chile?
Is $X < Y$?	`Yes`	Does Lota, Chile have smaller area than Buffalo, New York?
Is $Y < X$?	`No`	Does Buffalo, New York have smaller area than Lota, Chile?



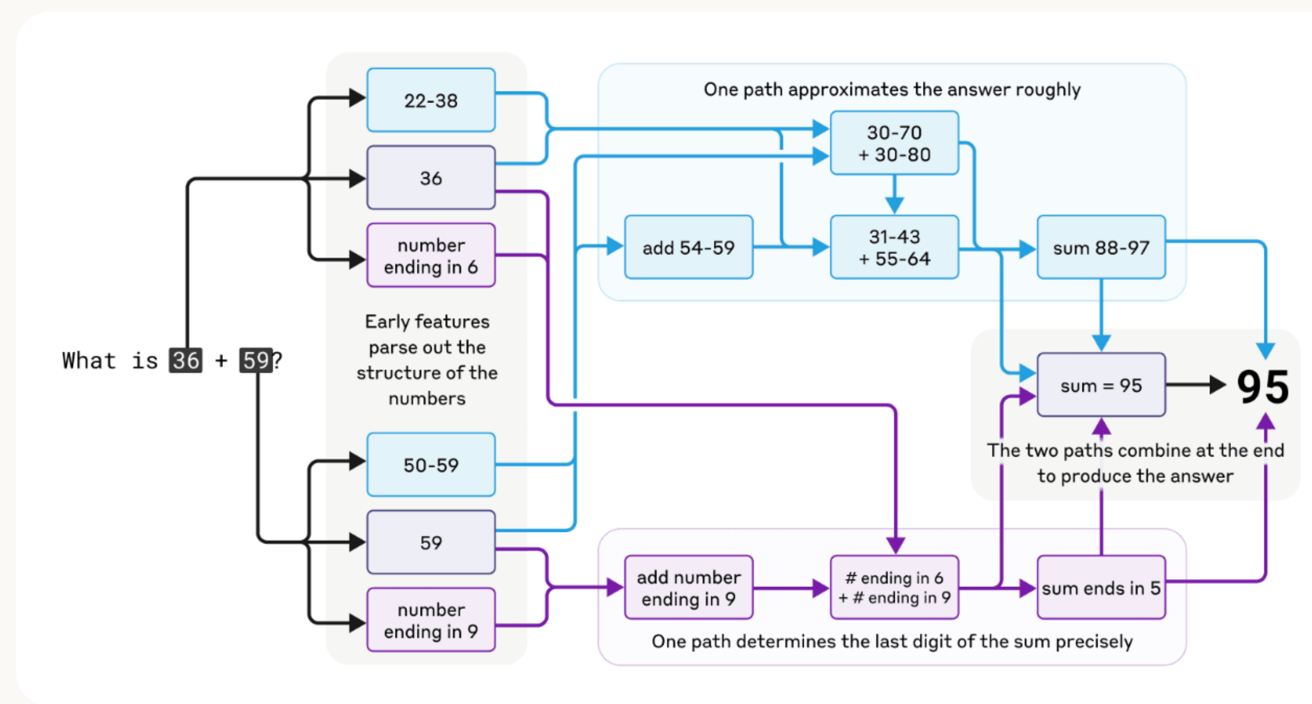
https://arxiv.org/pdf/2503.08679

Is CoT truly reflecting how the LLM "thinks"?

What is 36+59?



What the CoT says it is doing



The complex, parallel pathways in Claude's thought process while doing mental math.

What the model does under the hood

https://www.anthropic.com/research/tracing-thoughts-language-model

Bottom Line:

CoT is a window into plausible reasoning, not a reliable window into actual model computation.

But if we ignored this “terrifying” fact for now, we still want what the model verbally expressed to be correct.

Formal Pathway to Verification

Formal verification is basically:

- **Write the claim in a formal language**
- **Provide a derivation using only approved rules**
- **Let a verifier check each step mechanically**

So instead of asking:

- “Does this reasoning sound plausible?”

it asks:

- “Does every line follow from previous lines by a valid rule?”

That is where the guarantee comes from.

Simple Example

Suppose your proof state contains:

$$P$$

$$P \to Q$$

A human informally sees this and says: “then Q” … The LLM does the same thing.

But in a formal system, that move is **not automatic**. You can derive Q **only if your proof system includes a rule such as Implication Elimination**:

$$\frac{P \to Q \qquad P}{Q}$$

This rule explicitly authorizes the inference.

Formal Verification

In formal verification, conclusions follow only when an explicit inference rule licenses them.

In other words:

Premises = what is available
Rules = what transformations are permitted

A formal proof verifier does not reason by intuition.

So why are we not building formal verifiers?

- Requires task to be formalizable (i.e., impossible for many NLG problem)
- Real-world language is ambiguous, so converting it into a correct formal representation is difficult.
- It is often domain-specific and expensive, so it does not scale easily across open-ended tasks

(Has no alternative in critical domains where 100% correctness guarantees are needed)

Autoformalization tries to help with the scalability issue

1. Natural language problem
2. Translate to formal representation
3. Generate candidate proof / derivation
4. Verify with solver / theorem prover
5. Return answer only if verified

Autoformalization mostly helps with Translation step

Training a model to translate is relatively easy

We usually have training data to build a translation system – e.g.:

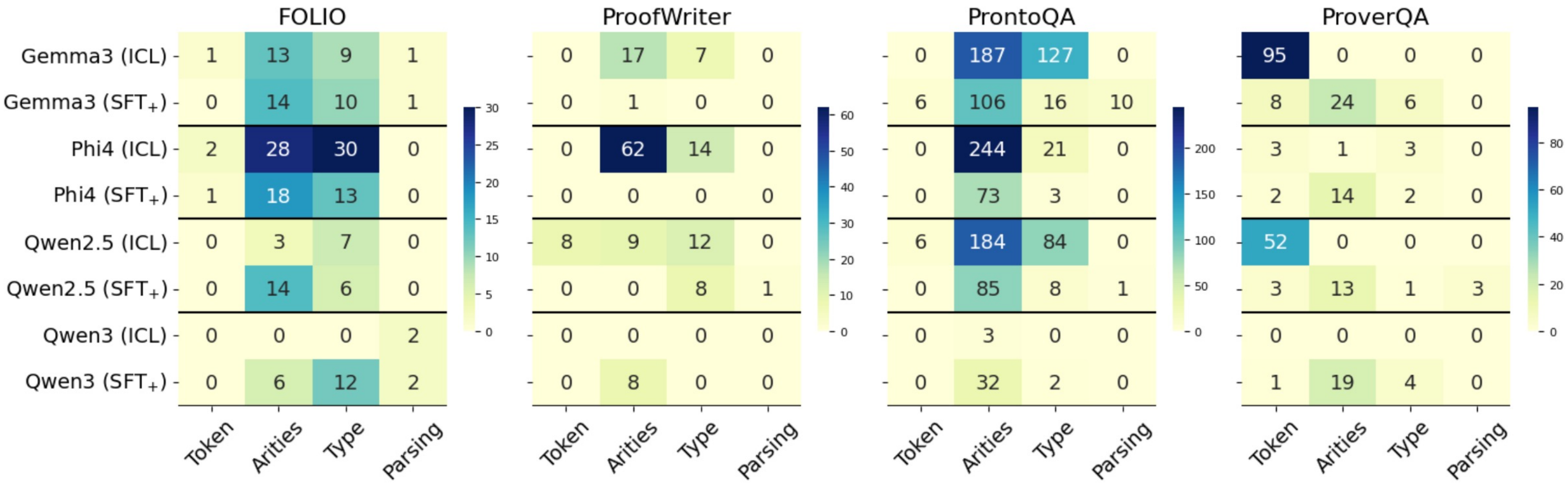
(**NATURAL LANGUAGE** "Every person is mortal." , **FOL:** **CORRECT FOL** $\forall x \; (Person(x) \rightarrow Mortal(x))$)

But translation of NL-to-Formal Language is error-prone process

	Type	Example
Parsing	Missing Quantifier	BerkeleyCollege(x) $\wedge$ResidentialCollegeAt(x, yaleUniversity)
	Parenthesis error	BeneficialTo(cherry, people) $\oplus$On(cherry, warningList)) $\rightarrow$ $\neg$RedFruit(cherry)
	Completion error	$\forall x$(Athlete(x) $\rightarrow\neg$NeverExercises(x)) Never: does not exist a time
Type	Quantifier location	$\exists y$(Own(emily, y) $\wedge$Roommate(y)) $\rightarrow$ $\exists y$(Own(emily, y) $\wedge$LiveIn(emily, apartment))
	Missing variable	$\forall x \exists y$(In(indonesia) $\wedge$ Prosecutor(x) $\wedge$ SpecialCrime(y) $\rightarrow$ InvestigatePersonally(x, y))
Token	Special token	Endowment(yale, 42.3 billion)
	Unknown Operator	$\forall x$(Rating(x, y) $\wedge$ $y \geq 4 \rightarrow$ Listed(x))
Predicate	Arity Mismatch	Sees(Tiger, Mouse); $\forall x$(((Visits(x, Rabbit)) $\wedge$ (Sees(Mouse))) $\rightarrow$ (Visits(x, Tiger)))
	Subject-Predicate Conflict	Platypus(platypus) $\wedge\neg$Teeth(platypus) $\wedge$Mammal(platypus)

This is just example of some syntax errors

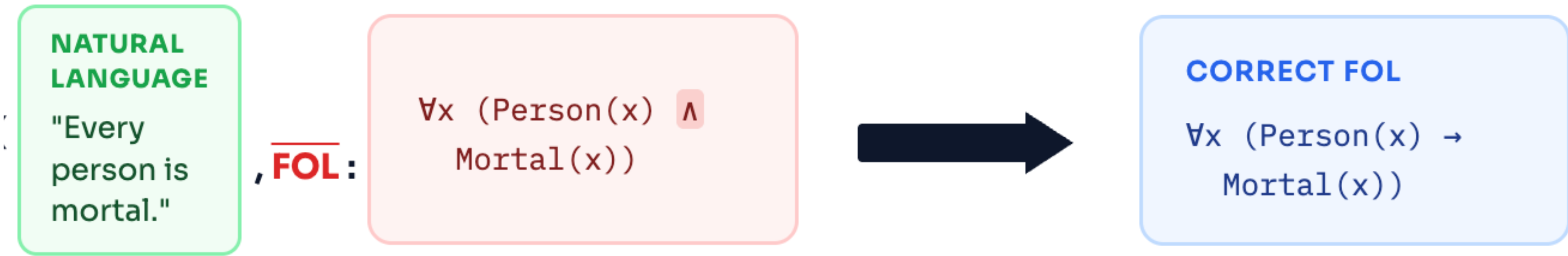
Leaving you with so many errors in the output



https://arxiv.org/pdf/2601.09446

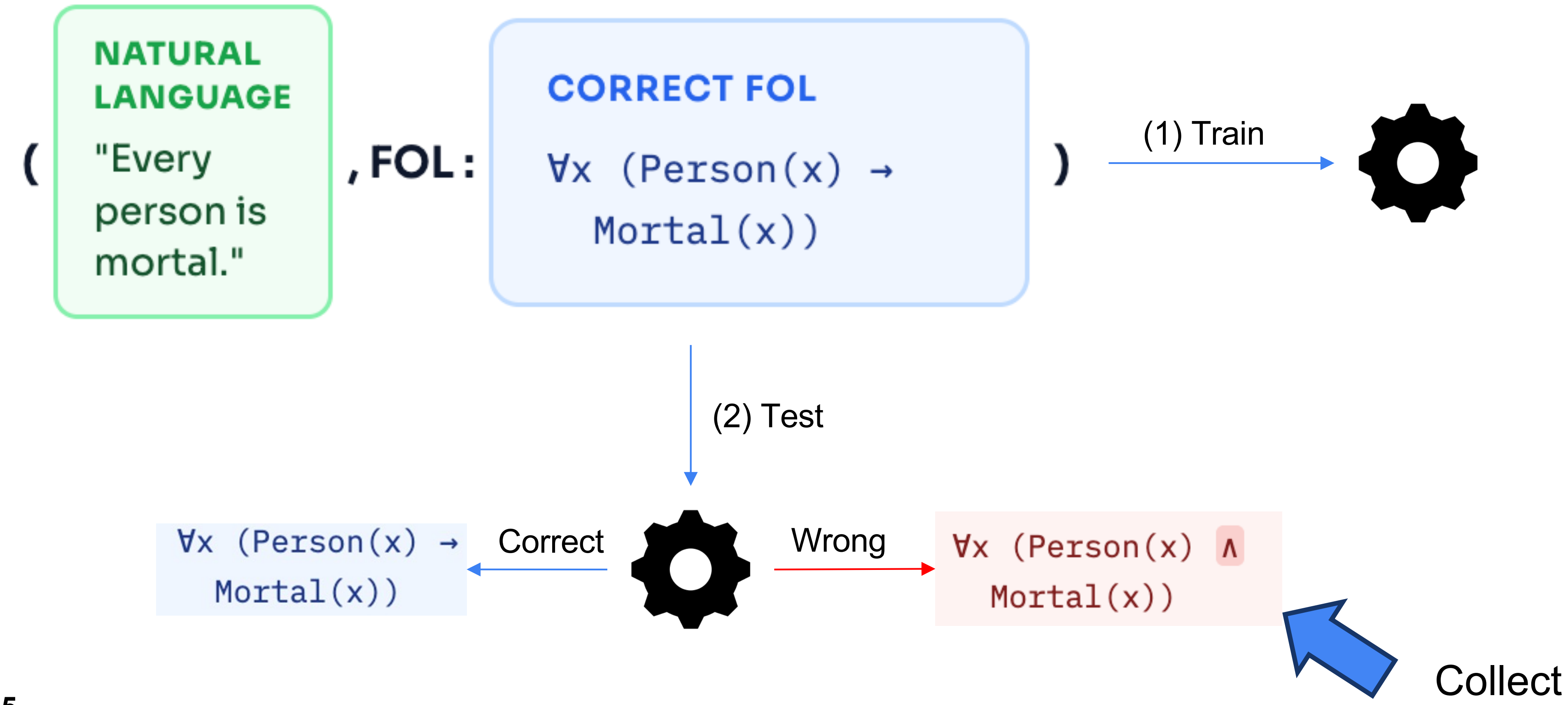
Training a model to spot the errors!

But the data for spotting/fixing error should look like this:



How do we collect this data?

(Idea 1)



(
NATURAL LANGUAGE
"Every person is mortal."
, FOL :
CORRECT FOL
∀x (Person(x) → Mortal(x))
)
(1) Train
(2) Test
∀x (Person(x) → Mortal(x))
Correct
Wrong
∀x (Person(x) ∧ Mortal(x))
Collect

(Idea 2)

(**NATURAL LANGUAGE** "Every person is mortal." , **FOL :** **CORRECT FOL** ∀x (Person(x) → Mortal(x)))

Perturb FOL

(e.g., replace → with ∧)

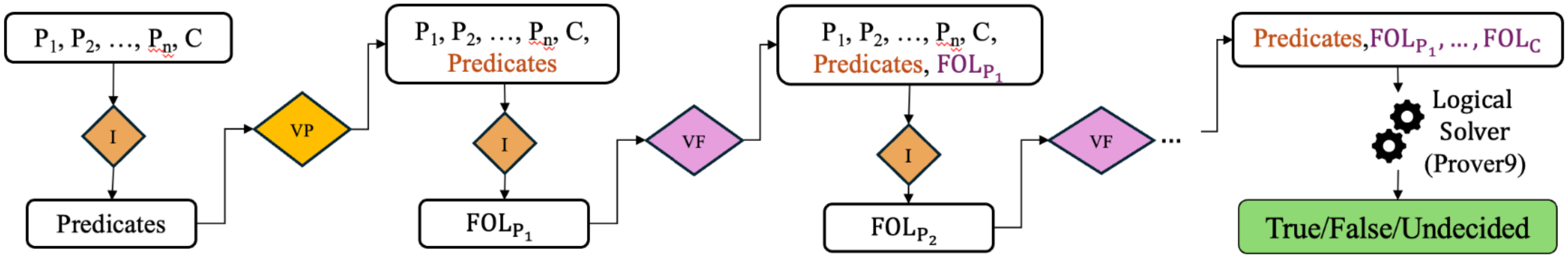
(**NATURAL LANGUAGE** "Every person is mortal." , **$\overline{\text{FOL}}$:** **PERTURBED FOL** ∀x (Person(x) ∧ Mortal(x)))

⚠ [→ changed to ∧]

Verifier Training Data = **Subset of (NL, FOL, "Correct") triples** from a seed NL-to-FOL dataset (i.e., given an NL sentence, generate FOL and label as "Correct").

\+ **Set of (NL, $\overline{\text{FOL}}$, Feedback) triples** for training the verifier (i.e., given an NL sentence and perturbed $\overline{\text{FOL}}$, generate Feedback identifying the logical error).

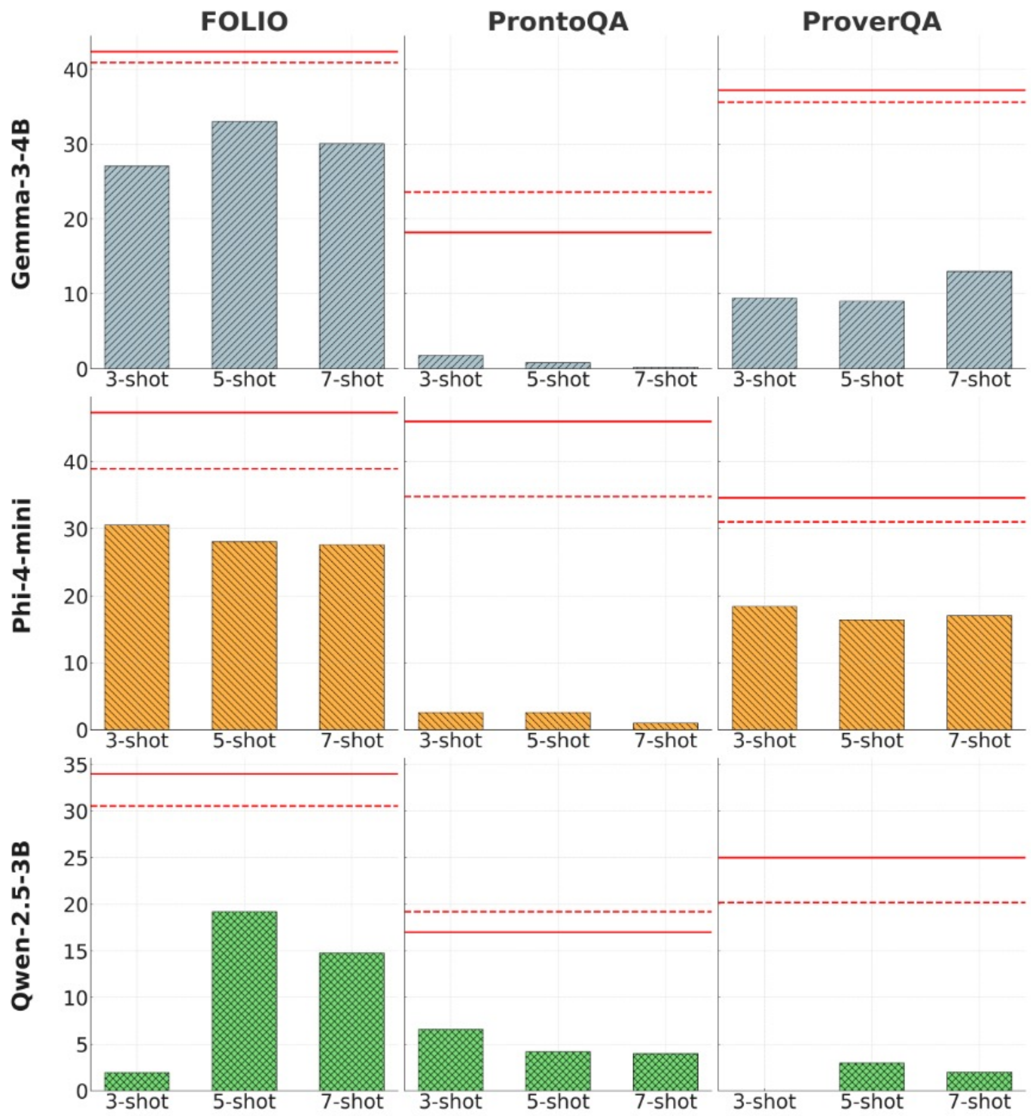
Inference time: The error detection/correction models



Generated predicates and FOLs are passed to a verifier (a specialised T5 model) to either confirm correctness or to apply the required error corrections before the next step of the translation.

https://arxiv.org/abs/2409.16461

Integration of translation corrector (solid red line), enhances reasoning accuracy substantially



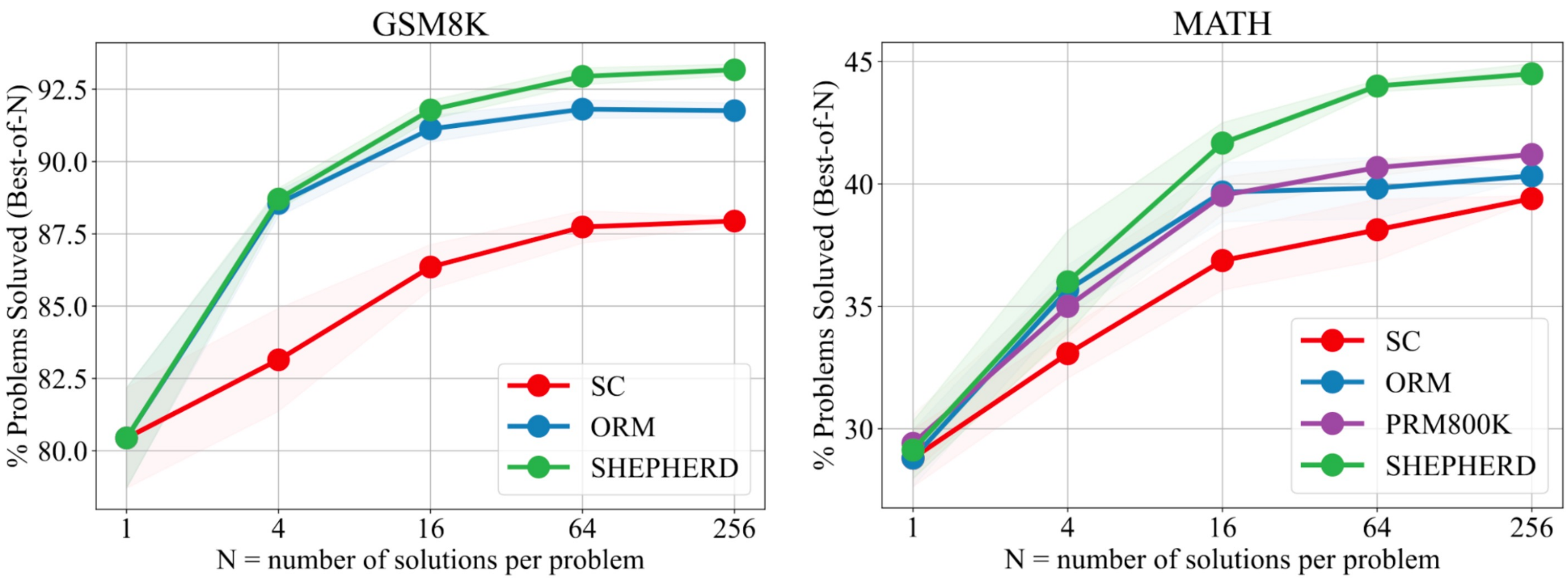
https://arxiv.org/abs/2601.09446

Bottom Line:

Formal pathway is the only way to get guarantees. But we know its limitations now. Also note, we only looked at FOL as formalism.

Next, we look at neural verifiers which do not have guarantees but are our 2nd best option.

Benefit of Neural Verifiers (inference-time scaling)



SC is majority voting

Why verification is even possible?

Verification vs. Generation Asymmetry

The key intuition is that checking a solution is often easier than producing one. This is a well-known asymmetry in computer science (P vs NP), verifying a proof is easier than discovering it.

Origin of Outcome-supervised reward mode / verifier - 2021

Training Verifiers to Solve Math Word Problems

Karl Cobbe* Vineet Kosaraju* Mohammad Bavarian Mark Chen
Heewoo Jun Łukasz Kaiser Matthias Plappert Jerry Tworek
Jacob Hilton Reiichiro Nakano Christopher Hesse John Schulman

OpenAI

https://arxiv.org/pdf/2110.14168

The introduction of GSM8K

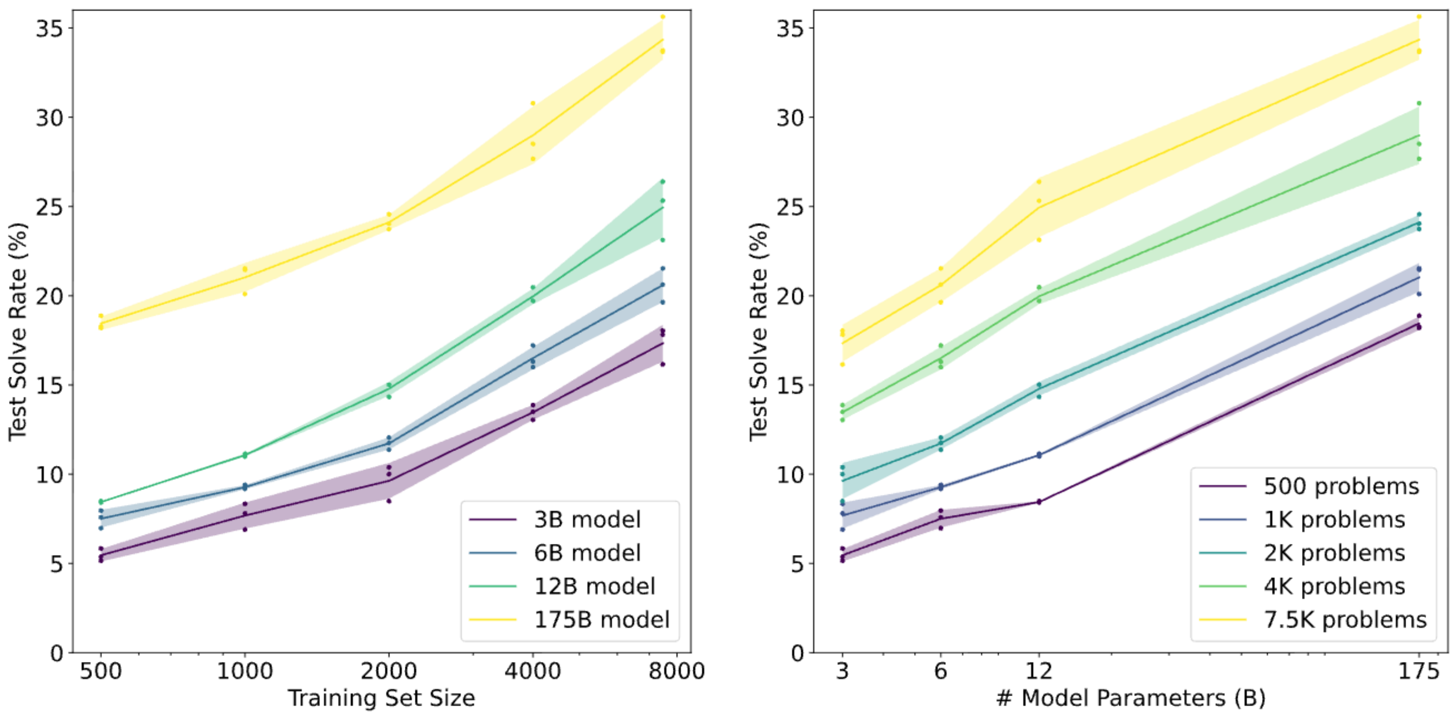
Problem: Beth bakes 4, 2 dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?
Solution: Beth bakes 4 2 dozen batches of cookies for a total of 4*2 = <<4*2=8>>8 dozen cookies
There are 12 cookies in a dozen and she makes 8 dozen cookies for a total of 12*8 = <<12*8=96>>96 cookies
She splits the 96 cookies equally amongst 16 people so they each eat 96/16 = <<96/16=6>>6 cookies
Final Answer: 6

Problem: Mrs. Lim milks her cows twice a day. Yesterday morning, she got 68 gallons of milk and in the evening, she got 82 gallons. This morning, she got 18 gallons fewer than she had yesterday morning. After selling some gallons of milk in the afternoon, Mrs. Lim has only 24 gallons left. How much was her revenue for the milk if each gallon costs $3.50?
Mrs. Lim got 68 gallons - 18 gallons = <<68-18=50>>50 gallons this morning.
So she was able to get a total of 68 gallons + 82 gallons + 50 gallons = <<68+82+50=200>>200 gallons.
She was able to sell 200 gallons - 24 gallons = <<200-24=176>>176 gallons.
Thus, her total revenue for the milk is $3.50/gallon x 176 gallons = $<<3.50*176=616>>616.
Final Answer: 616

Problem: Tina buys 3 12-packs of soda for a party. Including Tina, 6 people are at the party. Half of the people at the party have 3 sodas each, 2 of the people have 4, and 1 person has 5. How many sodas are left over when the party is over?
Solution: Tina buys 3 12-packs of soda, for 3*12= <<3*12=36>>36 sodas
6 people attend the party, so half of them is 6/2= <<6/2=3>>3 people
Each of those people drinks 3 sodas, so they drink 3*3=<<3*3=9>>9 sodas
Two people drink 4 sodas, which means they drink 2*4=<<4*2=8>>8 sodas
With one person drinking 5, that brings the total drank to 5+9+8+3= <<5+9+8+3=25>>25 sodas
As Tina started off with 36 sodas, that means there are 36-25=<<36-25=11>>11 sodas left
Final Answer: 11

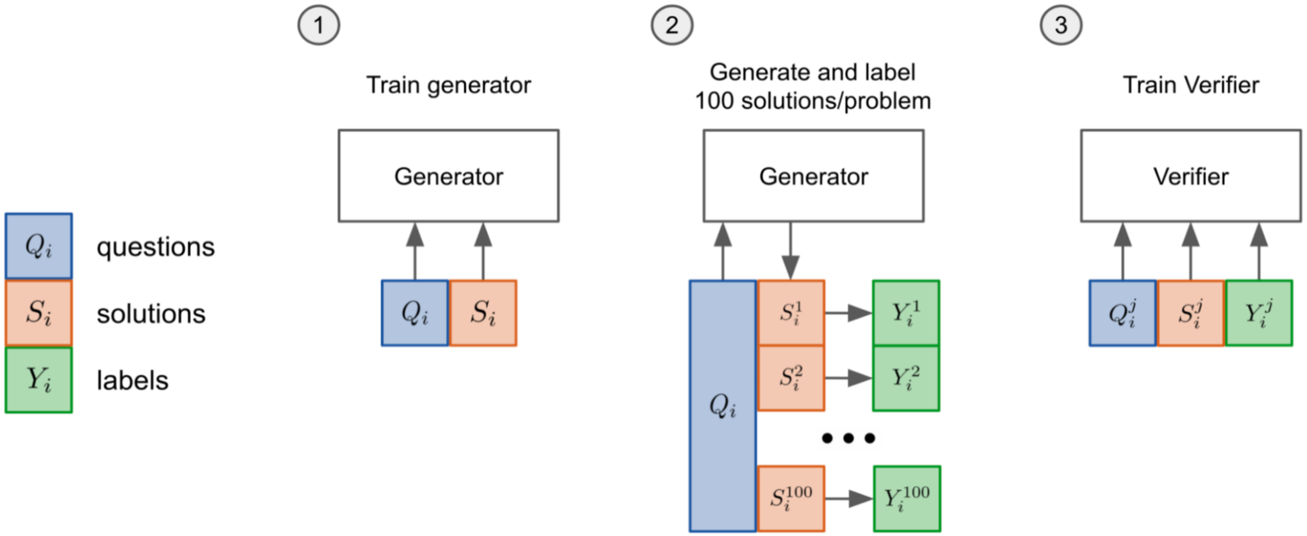
https://arxiv.org/pdf/2110.14168

Scaling Law – Note these LLMs are all early GPT-3 models



https://arxiv.org/pdf/2110.14168

ORM Verifier Training Pipeline

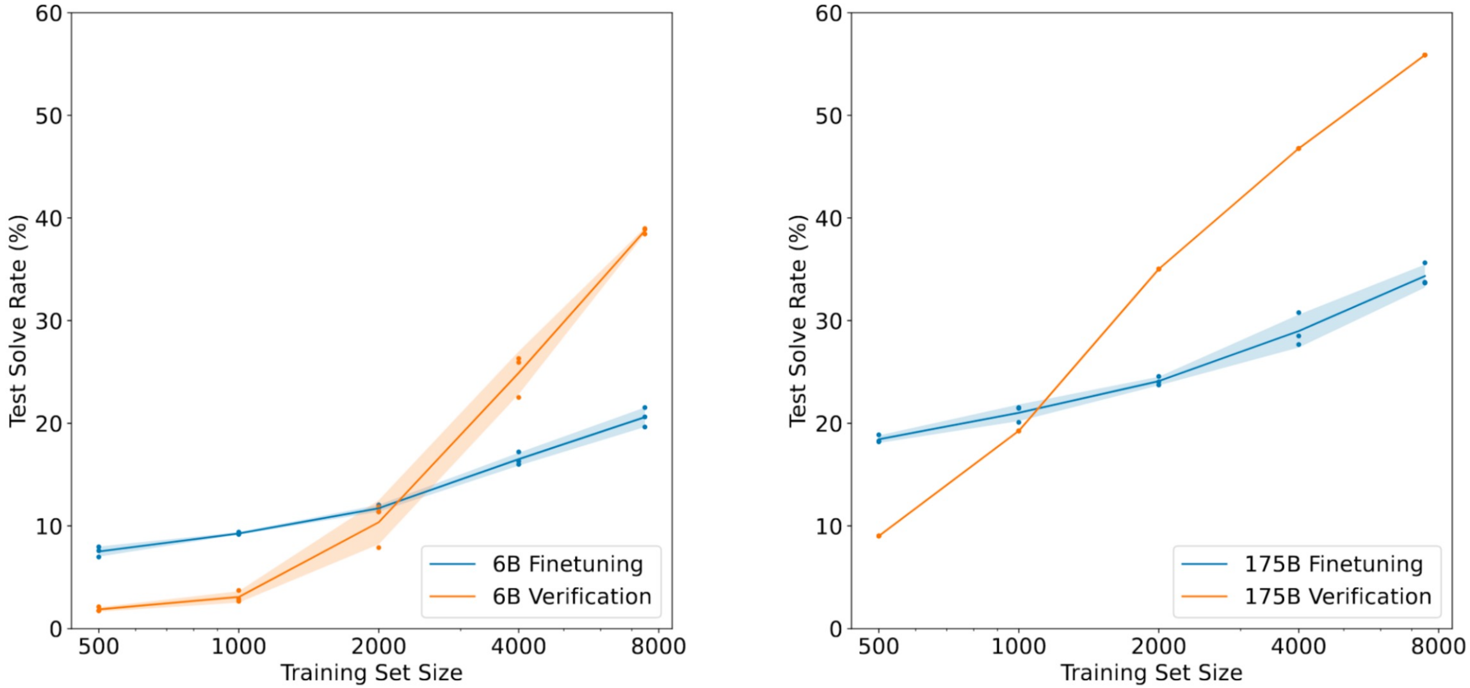


Trained the verifier as follows:

1. Finetune a model (the “generator / reasoner”) for 2 epochs on the training set.
2. Sample 100 completions from the generator for each training problem and label each solution as correct or incorrect.
3. Train a verifier for a single epoch on this dataset.

Verification vs. Fine-tuning

Conditioned on the problem and a candidate solution, the verifier outputs the **probability** that the solution is correct. At test time, we sample 100 completions to each test problem, **rank** them with the verifier, and then return the one with the **highest** verifier score.



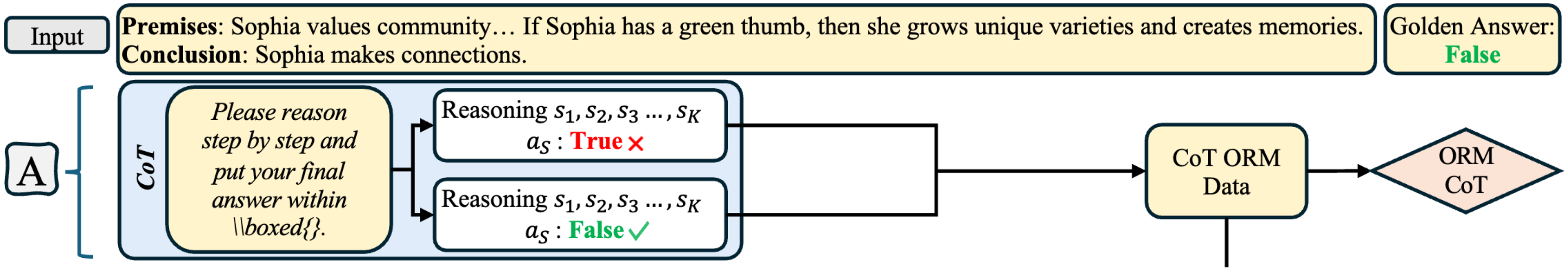
ORM is easy to train

Because all you need for training is the label of the final answer being correct or incorrect.

But, you still need to take care of a few things

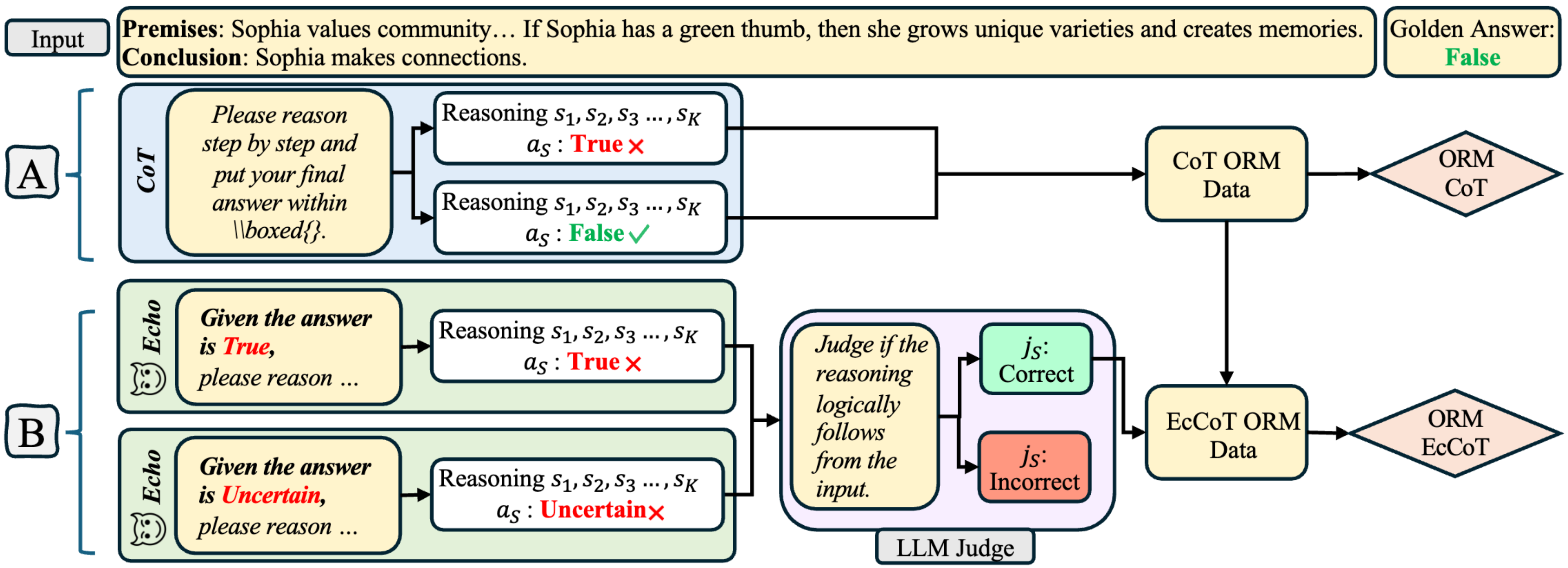
- Proportionality of Correct and Incorrect samples for training the verifier
- The coverage/diversity of errors the verifier is trained on
- And the generator-verifier dynamic (i.e., what if your verifier is a weaker/stronger model)?

Data for ORM



https://aclanthology.org/2025.emnlp-main.1326.pdf

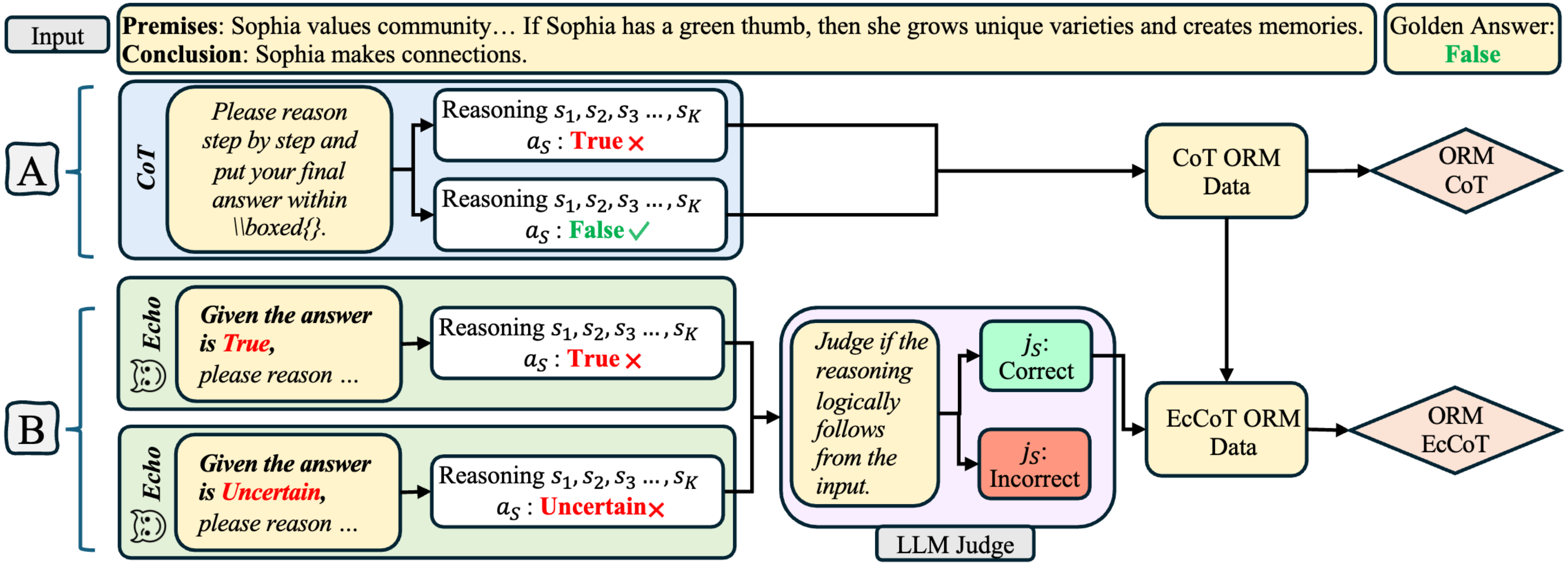
Data for ORM



https://aclanthology.org/2025.emnlp-main.1326.pdf

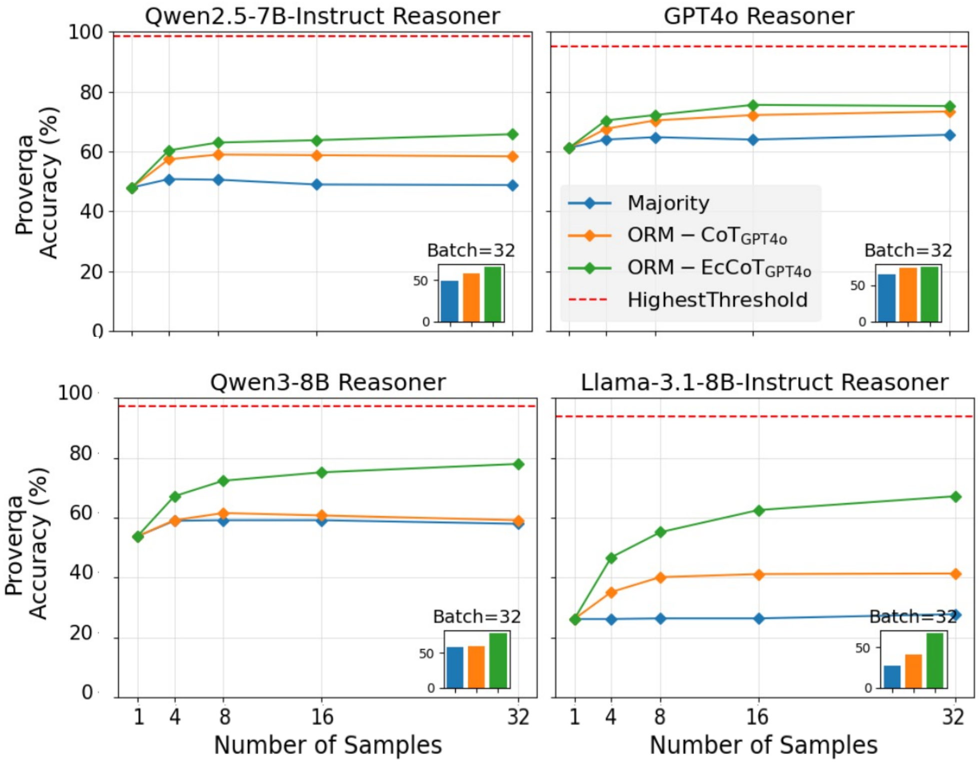
Errors by Echo

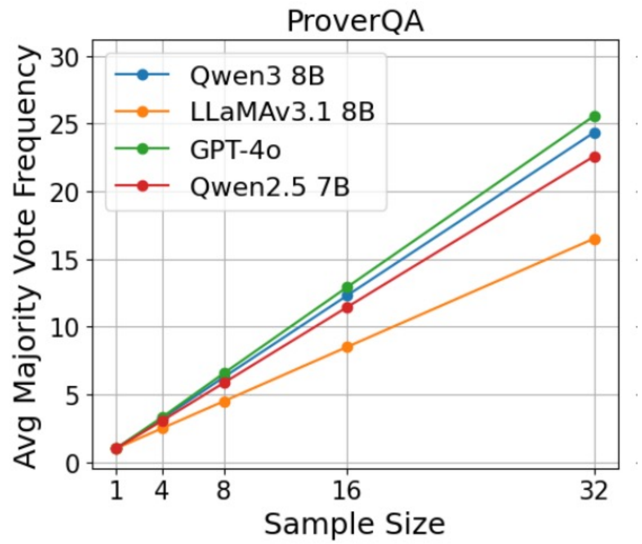
Mode	Echo	Total	Correct	Incorrect
CoT 0-shot	–	39998	34486 (86%)	5512 (14%)
Echo-CoT	–	63278	34486 (54%)	28792 (46%)



https://aclanthology.org/2025.emnlp-main.1326.pdf

ORM vs Majority Vote





If there is not much diversity in the sampled space, then ORM is less likely to help.

https://aclanthology.org/2025.emnlp-main.1326.pdf

Process-supervised Reward Model / verifier - 2023

Let's Verify Step by Step

Hunter Lightman* **Vineet Kosaraju*** **Yura Burda*** **Harri Edwards**

Bowen Baker **Teddy Lee** **Jan Leike** **John Schulman** **Ilya Sutskever**

Karl Cobbe*

OpenAI

https://arxiv.org/pdf/2305.20050

Human-annotated PRM Data

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to $2/5$, what is the numerator of the fraction? (Answer: $\boxed{14}$)

Let's call the numerator x.

So the denominator is 3x-7.

We know that x/(3x-7) = 2/5.

So 5x = 2(3x-7).

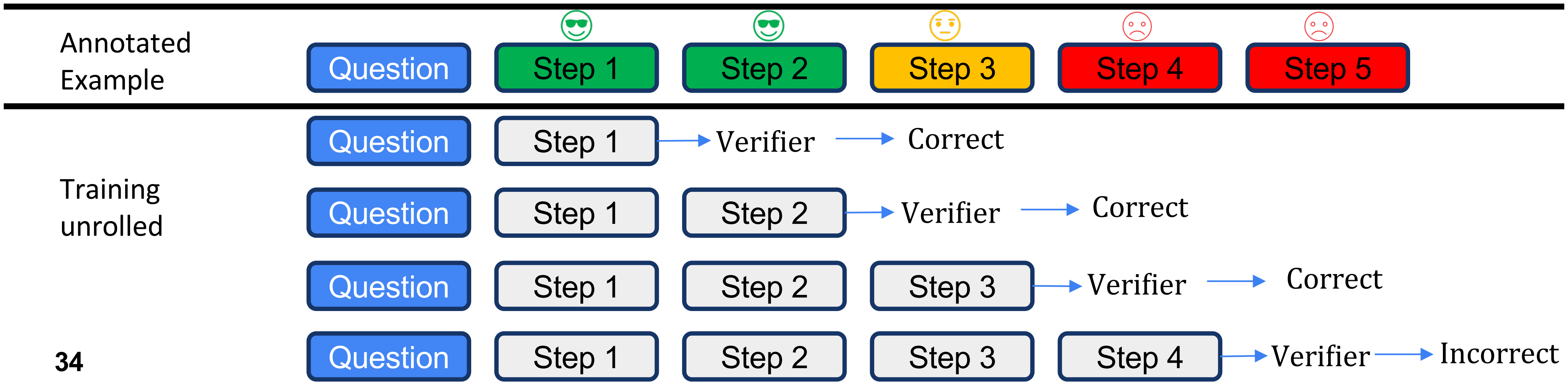
5x = 6x - 14.

So x = 7.

Training Algorithm

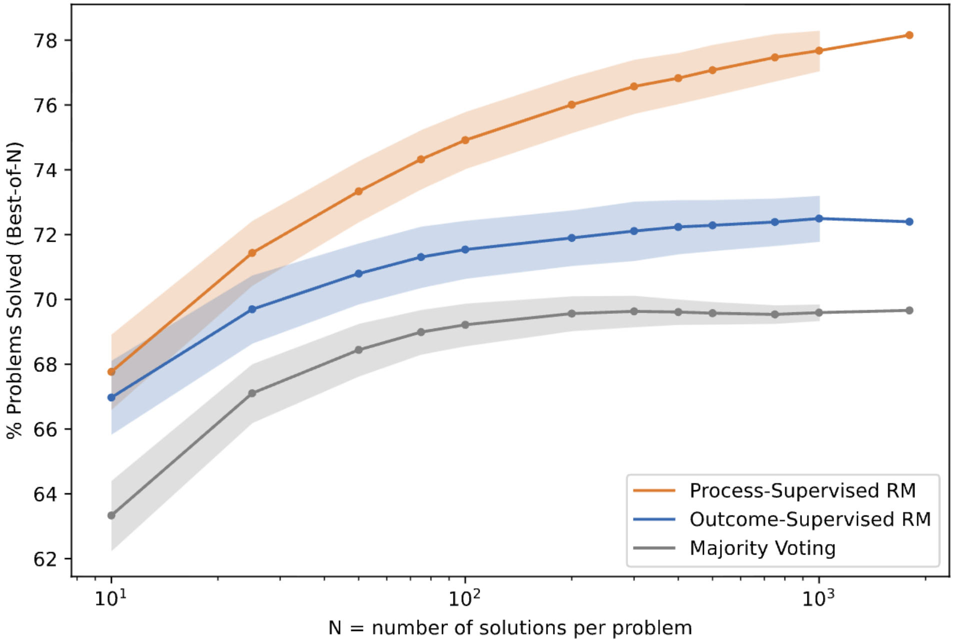
They train the PRM as a supervised language-model fine-tuning objective.

1. For each labeled solution, take **the solution prefix** up to the end of each labeled step.
2. The model predicts a single token representing the step label: positive, negative, neutral.
3. Training maximizes the log-likelihood of those target label tokens.



PRM vs ORM vs Majority Voting

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	**78.2**	69.6

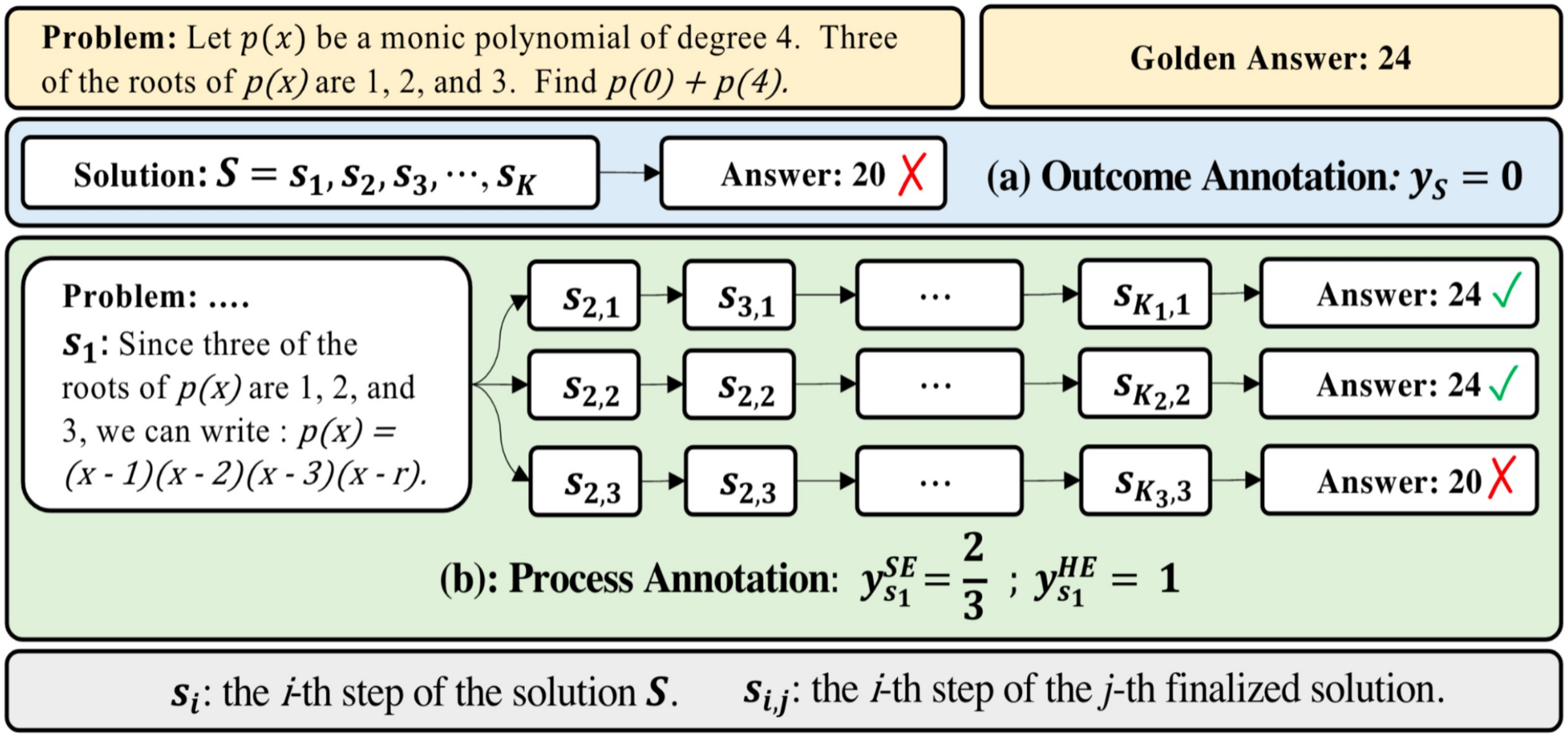


Result on MATH 500 test set

	ORM	PRM	Majority Vote
AP Calculus	68.9%	**86.7%**	80.0%
AP Chemistry	68.9%	**80.0%**	71.7%
AP Physics	77.8%	**86.7%**	82.2%
AMC10/12	49.1%	**53.2%**	32.8%
Aggregate	63.8%	**72.9%**	61.3%

OOD Result – 100 samples per problem

MATH-SHEPHERD: Automatic Process Annotation



https://aclanthology.org/2024.acl-long.510.pdf

Step Score Estimation

Estimation In this paper, we use two methods to estimate the quality y_{s_i} for the step s_i, hard estimation (HE) and soft estimation (SE). HE supposes that a reasoning step is good as long as it can reach the correct answer a^*:

$$y_{s_i}^{HE} = \begin{cases} 1 & \exists a_j \in A, a_j = a^* \\ 0 & \text{Otherwise} \end{cases} \tag{3}$$

SE assumes the quality of a step as the frequency with which it reaches the correct answer:

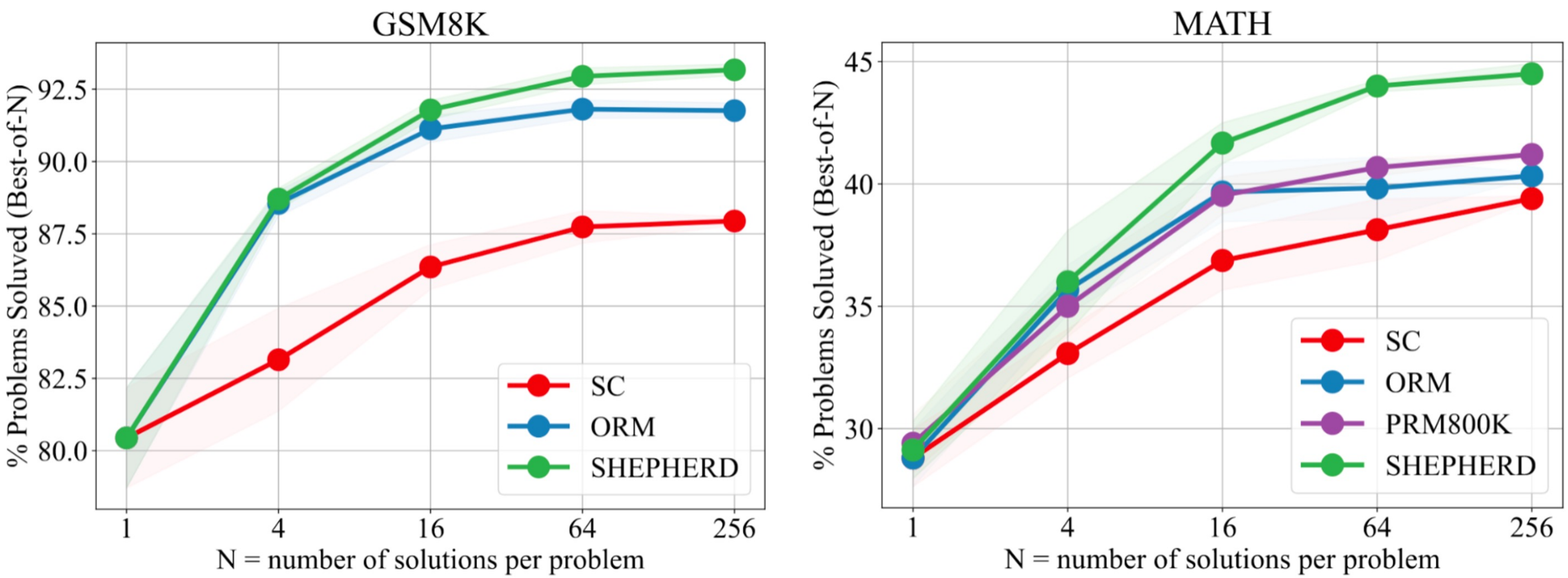
$$y_{s_i}^{SE} = \frac{\sum_{j=1}^{N} \mathbb{I}(a_j = a^*)}{N}. \tag{4}$$

Once we gather the label of each step, we can train PRM with the cross-entropy loss. In conclusion, our automatic process annotation framework defines the quality of a step as its potential to deduce the correct answer and achieve the label of each step by completion and estimation.

MATH-SHEPHERD PRM as Verifier

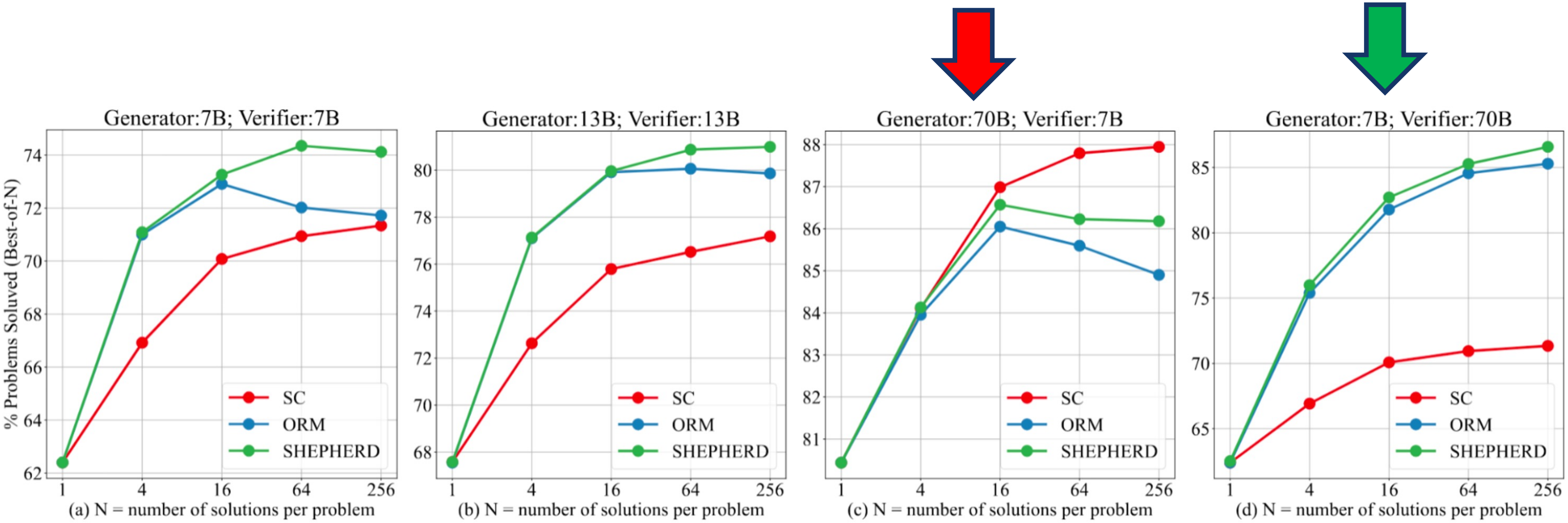
Models	Verifiers	GSM8K	MATH500
LLaMA2-70B: MetaMATH	Self-Consistency	88.0	39.4
	ORM	91.8	40.4
	Self-Consistency + ORM	92.0	42.0
	MATH-SHEPHERD (Ours)	93.2	44.5
	Self-Consistency + MATH-SHEPHERD (Ours)	92.4	45.2
LLemma-34B: MetaMATH	Self-Consistency	82.6	44.2
	ORM	90.0	43.7
	Self-Consistency + ORM	89.6	45.4
	MATH-SHEPHERD (Ours)	90.9	46.0
	Self-Consistency + MATH-SHEPHERD (Ours)	89.7	47.3
DeepSeek-67B: MetaMATH	Self-Consistency	88.2	45.4
	ORM	92.6	45.3
	Self-Consistency + ORM	92.4	47.0
	MATH-SHEPHERD (Ours)	**93.3**	47.0
	Self-Consistency + MATH-SHEPHERD (Ours)	92.5	**48.1**

Math-Shepherd vs PRM800K



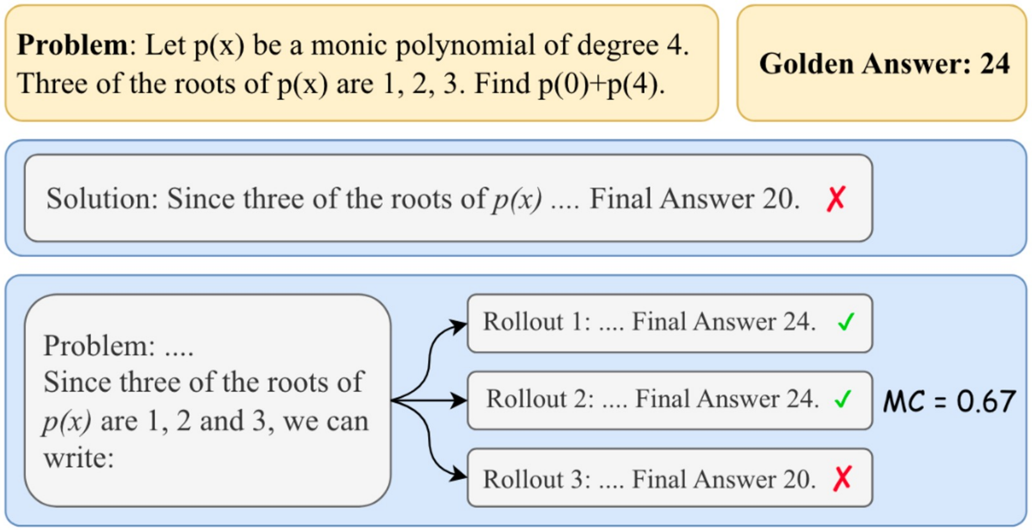
"PRM800K is annotated based on the output from GPT-4, and consequently, a discrepancy arises for the output of open-source LLaMA models fine-tuned on MetaMATH. Our dataset is four times larger than that provided in PRM800K."

Effect of Different Sizes of Generator and Verifier

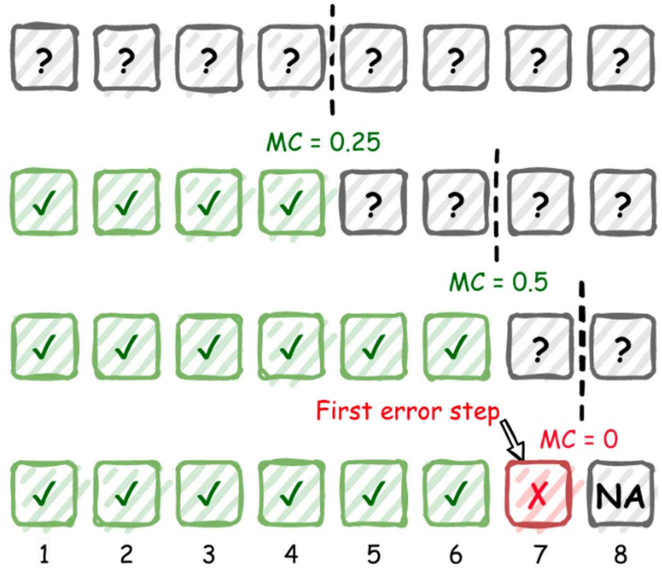


Regardless of data, you are still bounded by the verifier's backbone LLM capability gaps.

Improving Automatic Process Annotation Efficiency via Binary-search-based MC Estimation



(a) Monte Carlo estimation of a prefix solution.



(b) Error locating using binary search.

https://arxiv.org/pdf/2406.06592

Efficiency

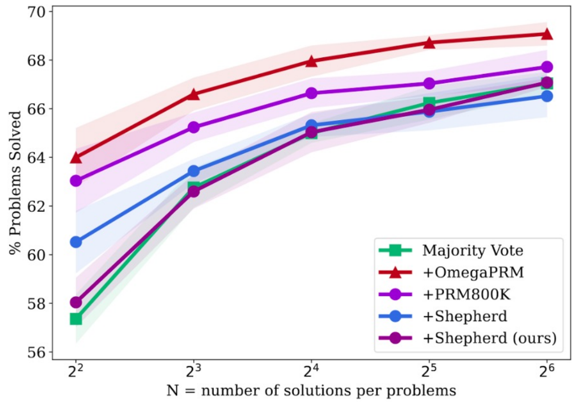
With the same computational budget:

Math-Shepherd was able to generate 200K data points using the brute-force algorithm, compared to 15 million data points with OmegaPRM, demonstrating a 75-times efficiency improvement.

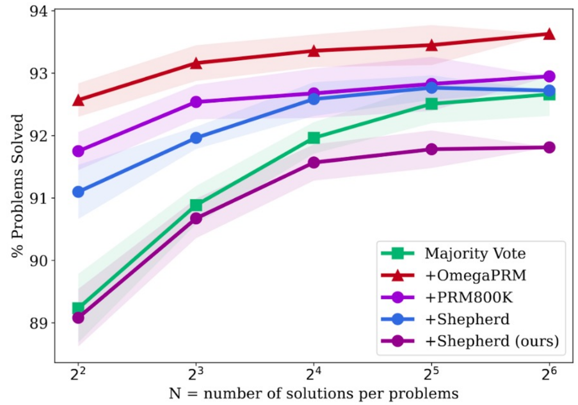
$$O(k \times M) \rightarrow O(k \times \log M)$$

M the length of the trace, and K the number of roll-outs

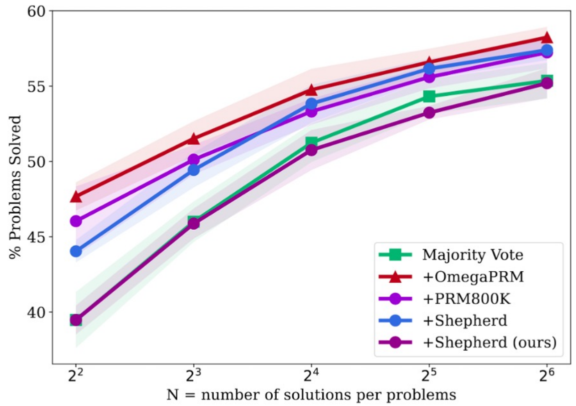
PRMs trained with different process supervision datasets



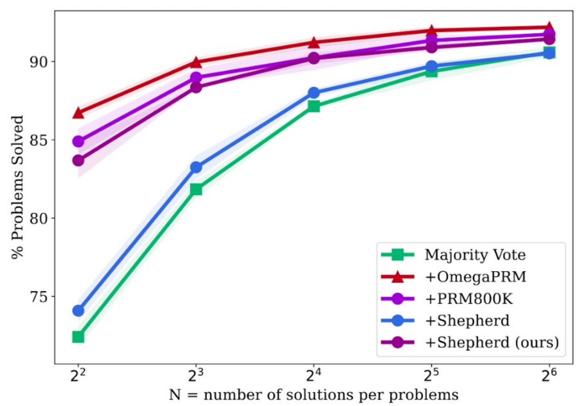
(a) Gemini Pro on MATH500.



(b) Gemini Pro on GSM8K.



(c) Gemma2 27B on MATH500.



(d) Gemma2 27B on GSM8K.

Testing directly on identifying errors

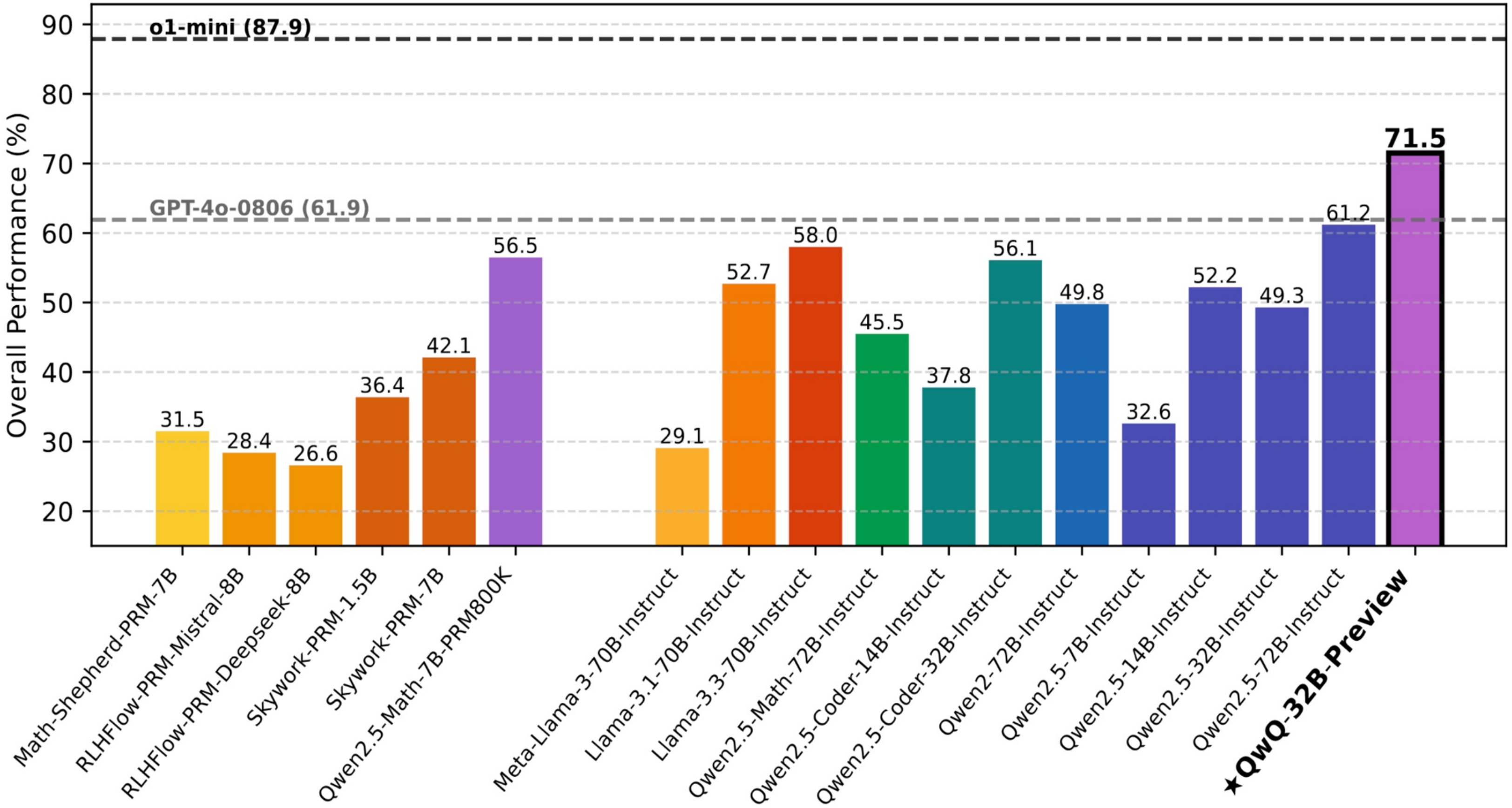
PROCESSBENCH: Identifying Process Errors in Mathematical Reasoning

Chujie Zheng Zhenru Zhang Beichen Zhang Runji Lin Keming Lu
Bowen Yu* Dayiheng Liu* Jingren Zhou Junyang Lin*
Qwen Team, Alibaba Inc.

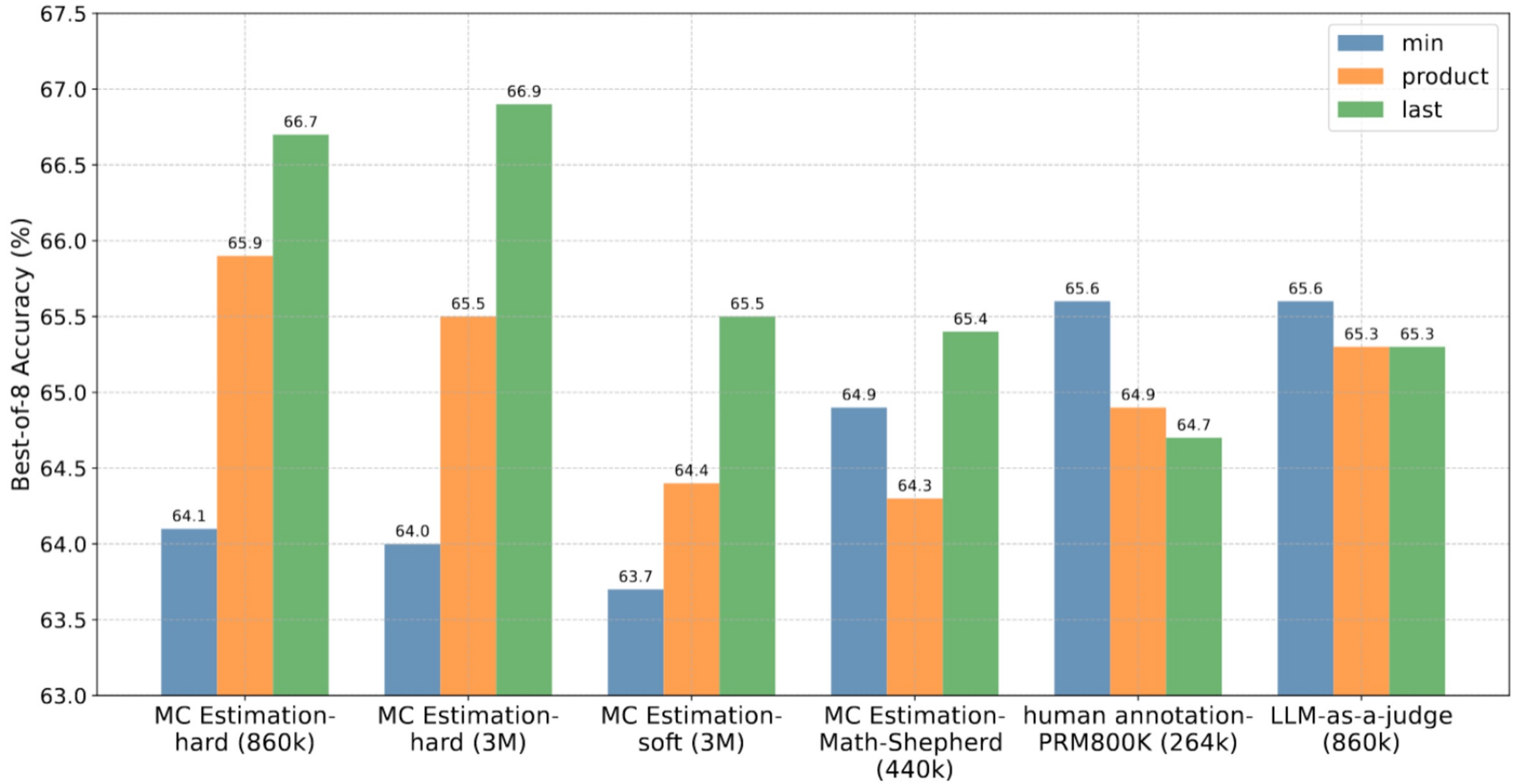
- Introduce PROCESSBENCH for measuring the ability to identify erroneous steps in mathematical reasoning.
- It consists of 3,400 test cases, primarily focused on competition- and Olympiad-level math problems.
- Each test case contains a step-by-step solution with error location annotated by human experts.

https://arxiv.org/pdf/2412.06559

Overview Results

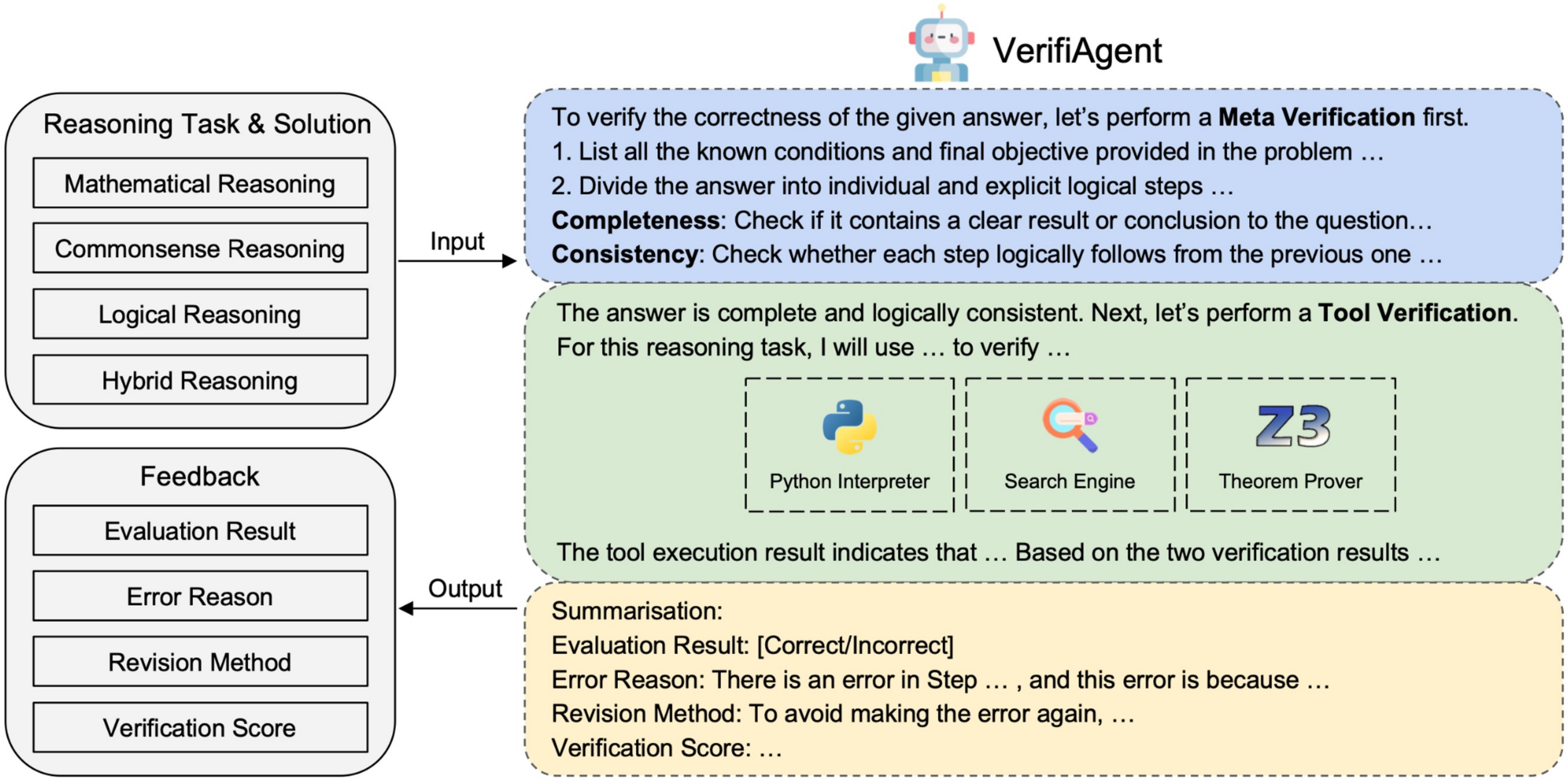


Different Scoring Methods for PRM-based selection



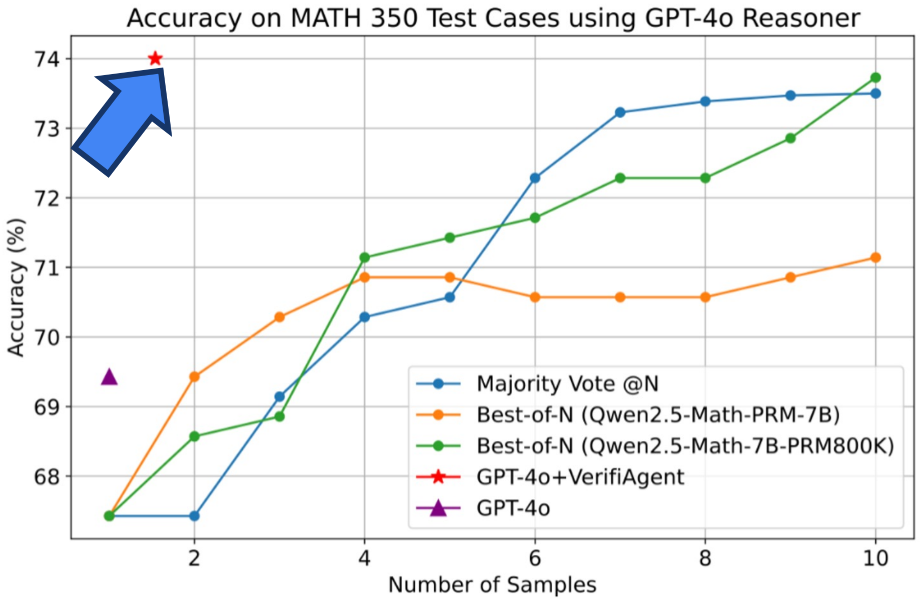
https://arxiv.org/pdf/2501.07301

Verifying Agent – training-free and multi-task

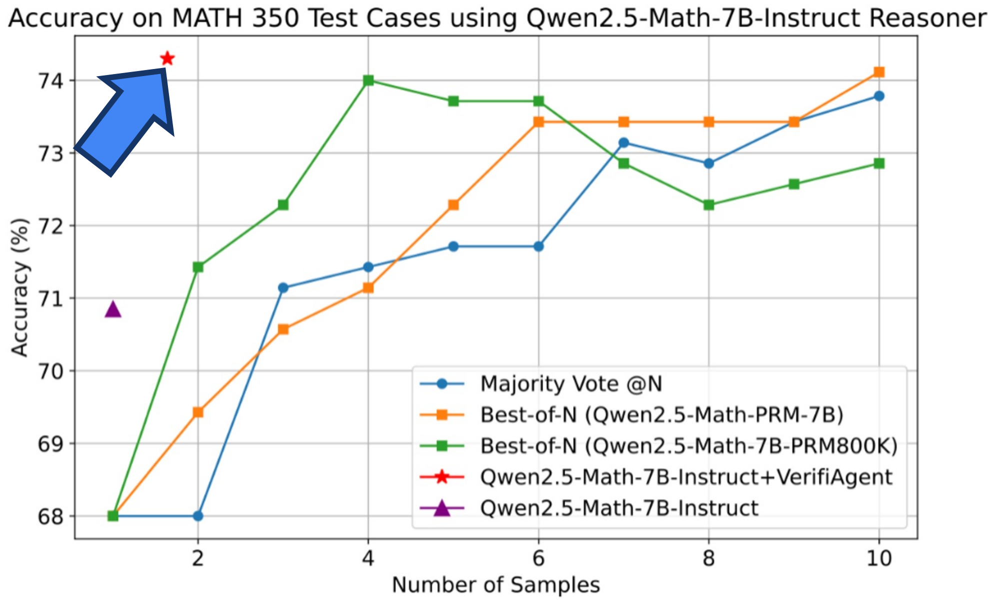


https://aclanthology.org/2025.findings-emnlp.891

Comparison to Inference Scaling with PRMs on MATH



(a) GPT-4o Reasoner



(b) Qwen2.5-Math-7B-Instruct Reasoner

The verifier is trained on traces from one model, how well does it score traces produced by another model?

The verifier learns the distribution/type of errors available in its training data.

The model used for generating/collecting these errors have its own reasoning capabilities, will make errors unique to itself.

The model used for generating/collecting these errors have its own style of reasoning (semantically and tactically).

The verifier is likely to only perform well in consistent settings.

Universality is hard to achieve.

Challenges/Open questions